

Mathematical Guide

All Models, Losses, Strategies, and Uncertainty Methods
in the AutoML Package

AutoML Research Project

April 2026 (rev. post Phase 8)

Contents

I	Framework	4
1	Introduction and Notation	4
1.1	Notation	4
2	Uncertainty Quantification: A Unified View	4
2.1	Taxonomy of Uncertainty Sources	5
2.2	Learned Variance via NLL	5
2.3	Law of Total Variance	5
2.4	MC Dropout	6
2.5	Binned Residual Standard Deviation	6
2.6	Summary: How Each Model Produces Uncertainty	6
II	Currently Supported Models	7
3	Linear Regression	7
4	Gradient-Boosted Trees (XGBoost, LightGBM, CatBoost)	7
4.1	Regularized Objective	7
4.2	Custom NLL Objective for Probabilistic Output	7
4.3	Implementation Differences	8
5	Plain PyTorch Neural Network	8
6	Classifier-Regression Model	8
6.1	Architecture Overview	8
6.2	Binning	8
6.3	Stage 1: Classification	9
6.4	Stage 2: Mapping	9
6.4.1	Linear Mapper	9
6.4.2	Lookup Mapper (LOOKUP_MEDIAN / LOOKUP_MEAN)	9
6.4.3	Spline Mapper	9
6.4.4	Neural Network Mapper	9

7	Probabilistic Regression Model (ProbReg)	9
7.1	Architecture Overview	9
7.2	Class Masking for Variable K	10
7.3	Regression Strategies	10
7.3.1	Strategy 1: Separate Heads (SEPARATE_HEADS)	10
7.3.2	Strategy 2: Single Head, N Outputs (SINGLE_HEAD_N_OUTPUTS)	11
7.3.3	Strategy 3: Single Head, Final Output (SINGLE_HEAD_FINAL_OUTPUT)	11
7.4	Law of Total Variance Aggregation	11
7.5	Gradient Stop Variant	11
7.6	Monotonic Constraints	11
7.7	Boundary Constraints	12
8	Flexible Neural Network (FlexibleNN)	12
8.1	Architecture	12
8.2	Soft Forward Pass	12
8.3	Hard Forward Pass (Inference)	13
8.4	Independent-Weights Variant	13
III	Dynamic Architecture Selection	14
9	Selection Strategies	14
9.1	None Strategy (Fixed)	14
9.2	Soft-Gating	14
9.3	Gumbel-Softmax	14
9.4	Straight-Through Estimator (STE)	14
9.5	REINFORCE	15
9.6	Strategy Summary	15
10	Regularization for Dynamic Selection	15
10.1	Penalty	15
10.2	ELBO with KL Divergence	15
10.3	Unified View: ELBO as Occam's Razor	15
10.4	Direct Regression Channel (ProbReg only)	16
IV	Loss Functions and Training	17
11	Loss Functions	17
11.1	Gaussian Negative Log-Likelihood (NLL)	17
11.2	β -NLL Loss (Seitzer et al., 2022)	17
11.3	Masked Cross-Entropy Loss	17
11.4	Combined Training Loss (ProbReg)	17
11.5	Boundary Regularization Loss	17
11.6	Middle-Class Penalty	17
11.7	Tree-Model NLL Objective	17
12	Learned Regularization	18
12.1	L1 Only (Laplace Prior)	18
12.2	L2 Only (Gaussian Prior)	18
12.3	L1 + L2 (Horseshoe-like Prior)	18

13 Target Transforms	18
13.1 Symmetric Log Transform (Symlog)	18
13.2 Uncertainty Conversion: Monte Carlo	18
14 Training Loop	18
 V Baseline Models	 20
15 NGBoost (Duan et al., 2020)	20
16 Mixture Density Network — MDN (Bishop, 1994)	20
17 Deep Ensembles (Lakshminarayanan et al., 2017)	20
18 Quantile Regression NN (Koenker, 2005)	20
19 Conformalized Quantile Regression (Romano et al., 2019)	21
20 Gaussian Process	21
21 Constant-Variance NN	21
22 FT-Transformer (Gorishniy et al., 2021)	21
23 TabPFN (Hollmann et al., 2024)	22
24 Laplace-Approximation BNN (Daxberger et al., 2021)	22
25 Conformal Prediction	22
25.1 Split Conformal (Marginal)	22
25.2 Locally-Adaptive Conformal (Lei & Wasserman, 2014)	22
 VI Evaluation	 23
26 Evaluation Metrics	23
26.1 Point-Prediction Metrics	23
26.2 Proper Scoring Rules	23
26.3 Interval Metrics	23
26.4 Calibration via PIT (Kuleshov et al., 2018)	23
26.5 Post-hoc Recalibration	24
26.6 PIT Uniformity Tests	24
27 Photometric Redshift Metrics	24

Part I

Framework

1 Introduction and Notation

This document provides a complete mathematical specification of every model, loss function, selection strategy, uncertainty method, and evaluation metric in the AutoML package. All sections describe currently implemented behavior unless explicitly flagged otherwise (e.g., Laplace-approximation BNN in §24).

1.1 Notation

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^d$	Input feature vector (d features)
$y \in \mathbb{R}$	Scalar regression target
N	Number of training samples
K	Number of classes / bins / mixture components
D	Number of hidden layers (depth) in FlexibleNN
M	Number of ensemble members or MC samples
$\pi_k(\mathbf{x})$	Probability of class k given input $\mathbf{x}$
$\mu_k(\mathbf{x})$	Mean prediction of regression head k
$\sigma_k^2(\mathbf{x})$	Variance prediction of regression head k
$\hat{y}(\mathbf{x})$	Final point prediction
$\hat{\sigma}^2(\mathbf{x})$	Final variance prediction
$\boldsymbol{\theta}$	All learnable parameters
$\boldsymbol{\phi}$	Parameters of a selection / gating subnetwork
τ	Temperature parameter (Gumbel-Softmax)
α	Miscoverage level for prediction intervals
β	Variance weighting exponent (β -NLL)
$k(\mathbf{x}, \mathbf{x}')$	Kernel / covariance function (GP)
η	Learning rate

Tensor shapes are written in parentheses: (B, K) means batch size B , K columns. We write $[n]$ for $\{1, 2, \dots, n\}$.

2 Uncertainty Quantification: A Unified View

Every model in this package produces (or can produce) an uncertainty estimate $\hat{\sigma}^2(\mathbf{x})$. This section provides the taxonomy *before* describing individual models, so the reader has the mental model up front.

2.1 Taxonomy of Uncertainty Sources

Method	How $\hat{\sigma}^2$ is produced	What it captures	Input-dep?
Learned variance (NLL)	Second output of NN, trained via NLL	Aleatoric	Yes
Law of Total Variance	Aggregation of per-class (μ_k, σ_k^2)	Aleatoric + epistemic-like	Yes
MC Dropout	Empirical variance of M stochastic passes	Epistemic (approx.)	Yes
Deep Ensembles	Variance across M independent models	Aleatoric + epistemic	Yes
Binned Residual Std	Variance of training residuals in bins	Aleatoric (discretized)	Partially
Lookup Mapper	Per-probability-bin variance	Aleatoric (discretized)	Partially
GP Posterior	Analytical posterior variance	Aleatoric + epistemic	Yes
Conformal	Calibration-set residual quantile	Coverage guarantee	No (marginal)
Constant	Global scalar, not learned	None (placeholder)	No

2.2 Learned Variance via NLL

The network outputs $(\mu(\mathbf{x}), \log \sigma^2(\mathbf{x}))$ and is trained with the Gaussian NLL (§11.1). The gradient w.r.t. $\log \sigma^2$:

$$\frac{\partial \mathcal{L}}{\partial \log \sigma^2} = \frac{1}{2} \left(1 - \frac{(y - \mu)^2}{\sigma^2} \right),$$

drives σ^2 toward $\mathbb{E}[(y - \mu)^2 \mid \mathbf{x}]$, the true conditional variance, at convergence.

Used by: ProbReg (per-head), Plain NN (**PROBABILISTIC**), NN Mapper, Deep Ensembles (per member), NGBoost, tree models (custom objective).

2.3 Law of Total Variance

Given per-class predictions $\{(\mu_k, \sigma_k^2)\}_{k=1}^K$ weighted by $\{\pi_k\}_{k=1}^K$:

$$\hat{\sigma}^2 = \underbrace{\sum_k \pi_k \sigma_k^2}_{\text{expected within-class variance}} + \underbrace{\sum_k \pi_k (\mu_k - \hat{y})^2}_{\text{between-class variance}}. \quad (1)$$

The first term is aleatoric (noise within each class). The second term captures disagreement among class means. When all μ_k agree, the second term vanishes and we get pure aleatoric uncertainty.

Used by: ProbReg (central novelty), MDN, Deep Ensembles (with uniform $\pi_m = 1/M$).

2.4 MC Dropout

Gal & Ghahramani (2016): keep dropout active at test time and run M forward passes:

$$\hat{y}_m = f_{\theta, \mathbf{z}_m}(\mathbf{x}), \quad \mathbf{z}_m \sim \text{Bernoulli}(1 - p)^L, \quad m = 1, \dots, M, \quad (2)$$

$$\hat{\mu} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m, \quad \hat{\sigma}^2 = \frac{1}{M} \sum_{m=1}^M (\hat{y}_m - \hat{\mu})^2. \quad (3)$$

Used by: Plain NN (with MC_DROPOUT).

2.5 Binned Residual Standard Deviation

A post-hoc method. After training:

1. Compute residuals: $r_i = y_i - \hat{y}_i$ for all training samples.
2. Bin predictions into B bins by $\hat{y}_i$.
3. For each bin b : $\hat{\sigma}_b = \text{std}(\{r_i : \hat{y}_i \in b\})$.
4. At test time: $\hat{\sigma}(\mathbf{x}) = \hat{\sigma}_{b(\hat{y}(\mathbf{x}))}$.

Partially input-dependent: piecewise constant (one value per bin).

Used by: Linear Regression, XGBoost, LightGBM, CatBoost, Plain NN (with BINNED_RESIDUAL_STD).

2.6 Summary: How Each Model Produces Uncertainty

Model	Primary UQ Method	Alternatives
Linear Regression	Binned Residual Std	—
Plain NN	Depends on <code>uncertainty_method</code>	CONSTANT / PROBABILISTIC / MC_DROPOUT /
XGBoost	Custom NLL objective (learned σ^2)	Binned Residual Std
LightGBM	Custom NLL objective (learned σ^2)	Binned Residual Std
CatBoost	Custom NLL objective (learned σ^2)	Binned Residual Std
ProbReg	Law of Total Variance	+ Conformal wrapper
ClassReg (Lookup)	Per-bin variance from lookup table	Conformal wrapper
ClassReg (Spline)	Constant residual variance	—
ClassReg (NN Mapper)	Learned σ^2 via NLL	—
FlexibleNN	Same as Plain NN	—
GP	Posterior variance (analytical)	—
Deep Ensembles	Ensemble disagreement + mean aleatoric	—
NGBoost	Natural gradient learned σ^2	—
MDN	Mixture variance (law of total variance)	—
Quantile Regression	Quantile-based intervals	—
CQR	Quantile regression + conformal calibration	—

Part II

Currently Supported Models

3 Linear Regression

Ordinary least squares:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad (\mathbf{w}, b) = \arg \min_{\mathbf{w}, b} \sum_{i=1}^N (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2.$$

Uncertainty: No native uncertainty. Uses post-hoc BINNED_RESIDUAL_STD (§2.5).

4 Gradient-Boosted Trees (XGBoost, LightGBM, CatBoost)

All three share the same additive ensemble structure:

$$\hat{y}(\mathbf{x}) = \sum_{t=1}^T \eta \cdot f_t(\mathbf{x}), \quad f_t \in \mathcal{F},$$

where $\mathcal{F}$ is the space of decision trees, η is the learning rate (shrinkage), and each tree f_t is fit to the pseudo-residuals.

4.1 Regularized Objective

At step t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^N \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma T_{\text{leaves}} + \frac{1}{2} \lambda \sum_{j=1}^{T_{\text{leaves}}} w_j^2, \quad (4)$$

where $g_i = \partial \mathcal{L} / \partial \hat{y}_i$ and $h_i = \partial^2 \mathcal{L} / \partial \hat{y}_i^2$.

4.2 Custom NLL Objective for Probabilistic Output

For trees predicting $(\mu, \log \sigma^2)$, custom gradients and Hessians:

Mean output:

$$g_\mu = \frac{\mu - y}{\sigma^2}, \quad h_\mu = \frac{1}{\sigma^2}. \quad (5)$$

Log-variance output:

$$g_{\log \sigma^2} = \frac{1}{2} \left(1 - \frac{(y - \mu)^2}{\sigma^2} \right), \quad h_{\log \sigma^2} = \frac{1}{2} \quad (\text{Fisher information}). \quad (6)$$

The Fisher information $h = \frac{1}{2}$ is preferred over the observed Hessian $\frac{1}{2}(y - \mu)^2 / \sigma^2$ because it is always positive and gives more stable tree splits.

4.3 Implementation Differences

- **XGBoost** (Chen & Guestrin, 2016): exact or approximate greedy split finding, L1/L2 regularization on leaf weights.
 - **LightGBM** (Ke et al., 2017): histogram-based splits, gradient-based one-side sampling (GOSS), exclusive feature bundling (EFB).
 - **CatBoost** (Prokhorenkova et al., 2018): ordered boosting to prevent target leakage, oblivious (symmetric) trees with balanced splits at each depth.
- Uncertainty:** Learned $\sigma^2(\mathbf{x})$ via custom NLL objective, or post-hoc BINNED_RESIDUAL_STD.

5 Plain PyTorch Neural Network

A feedforward MLP:

$$f_{\theta}(\mathbf{x}) = W_L \cdot \sigma(W_{L-1} \cdots \sigma(W_1 \mathbf{x} + b_1) \cdots + b_{L-1}) + b_L,$$

where σ is an activation function (ReLU, GELU, etc.) and L is the number of layers.

Uncertainty modes:

- **CONSTANT:** No learned uncertainty. $\hat{\sigma}^2$ is a global constant.
- **PROBABILISTIC:** Output layer produces $(\mu, \log \sigma^2)$; trained with Gaussian NLL (§2.2).
- **MC_DROPOUT:** Dropout active at test time; run M forward passes (§2.4).
- **BINNED_RESIDUAL_STD:** Post-hoc binned variance (§2.5).

6 Classifier-Regression Model

A two-stage model: (1) discretize the target into bins and train a classifier, (2) map classifier probabilities back to regression predictions.

6.1 Architecture Overview

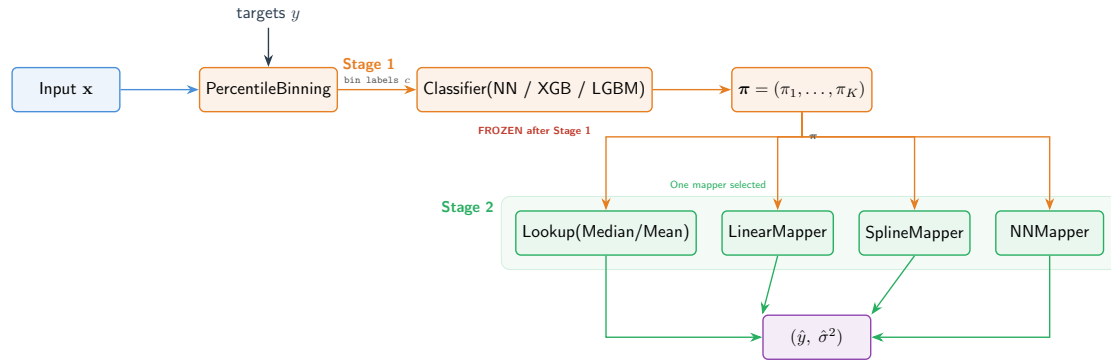


Figure 1: ClassifierRegression two-stage architecture. The classifier is frozen before Stage 2 (mapper) training.

6.2 Binning

Targets are binned using percentile-based boundaries:

$$q_j = \text{Percentile} \left(y_{\text{train}}, \frac{100 \cdot j}{K} \right), \quad j = 0, 1, \dots, K, \quad (7)$$

$$c_i = \max\{j : y_i \geq q_j\}, \quad c_i \in \{0, 1, \dots, K-1\}. \quad (8)$$

Duplicate boundaries (from repeated values) are collapsed, and K is reduced accordingly.

6.3 Stage 1: Classification

Train a classifier $f : \mathbb{R}^d \rightarrow \Delta^K$ (NN, XGBoost, LightGBM, or CatBoost) on $(\mathbf{x}_i, c_i)$ pairs. After training, **freeze** the classifier.

6.4 Stage 2: Mapping

A *mapper* transforms class probabilities to regression predictions.

6.4.1 Linear Mapper

$\hat{y} = a \cdot \hat{\pi} + b$, where $\hat{\pi}$ is a scalar summary of the probability vector and (a, b) are fit by OLS.

Uncertainty: Constant residual variance $\hat{\sigma}^2 = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$.

6.4.2 Lookup Mapper (LOOKUP_MEDIAN / LOOKUP_MEAN)

The most reliable mapper. Bins the probability space and computes per-bin statistics:

1. For each class k , collect $\pi_k(\mathbf{x}_i)$ across training samples.
2. Bin the probability values into B bins (where $B = \max(B_{\min}, \min(B_{\max}, N/100))$).
3. For each bin b :

$$\hat{y}_b = \text{median}(\{y_i : \pi_k(\mathbf{x}_i) \in b\}) \quad (\text{or mean}), \quad (9)$$

$$\hat{\sigma}_b^2 = \text{var}(\{y_i : \pi_k(\mathbf{x}_i) \in b\}) \quad (\text{if PROBABILISTIC}). \quad (10)$$

4. At prediction time: look up the bin for $\pi_k(\mathbf{x})$, return $(\hat{y}_b, \hat{\sigma}_b^2)$.

Uncertainty: Per-probability-bin variance — *input-dependent* through the probability: different π_k values land in different bins.

6.4.3 Spline Mapper

$\hat{y} = S(\pi_k; \text{degree} = k_s, \text{smoothing} = s)$ via `UnivariateSpline`. Known to be unstable.

Uncertainty: Constant residual variance (no per-input variance).

6.4.4 Neural Network Mapper

$(\hat{y}, \log \hat{\sigma}^2) = h_{\theta(\pi)}$ where h is a small MLP. Trained with NLL loss (probabilistic) or MSE (deterministic).

Uncertainty: Learned input-dependent $\hat{\sigma}^2$ via NLL. Known training issues due to two-stage optimization.

7 Probabilistic Regression Model (ProbReg)

The central idea: discretize the prediction task into K classes via a learned classifier, attach a per-class regression head that outputs $(\mu_k, \log \sigma_k^2)$, and combine via the **law of total variance**.

7.1 Architecture Overview

1. **Classifier backbone** $f_{\theta: \mathbb{R}^d \rightarrow \mathbb{R}^K}$. Produces raw logits $\ell_1, \dots, \ell_K$. Softmax gives class probabilities:

$$\pi_k(\mathbf{x}) = \frac{\exp(\ell_k(\mathbf{x}))}{\sum_{j=1}^K \exp(\ell_j(\mathbf{x}))}.$$

2. **Regression module** (one of three strategies, §7.3).

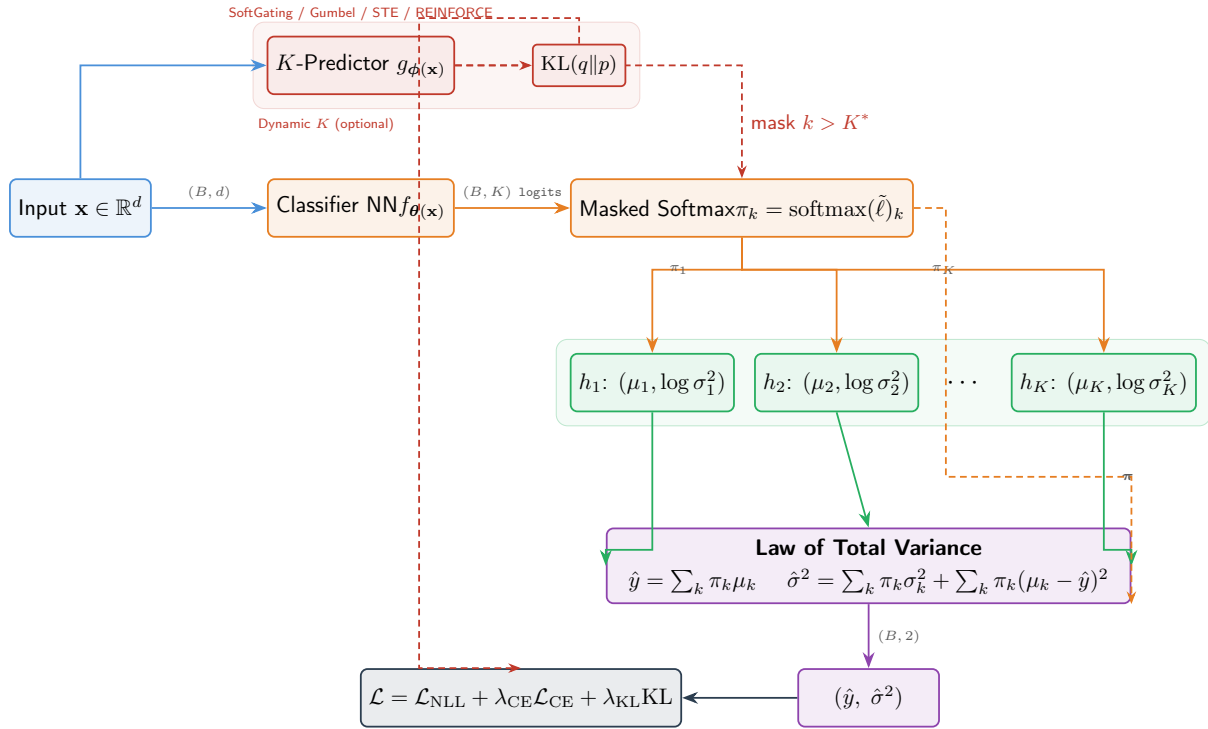


Figure 2: ProbReg architecture. Solid arrows: data flow. Dashed: optional dynamic- K path. Each regression head h_k receives only its own probability π_k (SEPARATE_HEADS strategy).

3. **Aggregation** via law of total variance (§2.3).
4. **Optional: dynamic K** via a gating subnetwork (§9).

7.2 Class Masking for Variable K

When using dynamic K or a fixed $K < K_{\max}$, classes $k > K$ are masked:

$$\tilde{\ell}_k = \begin{cases} \ell_k & \text{if } k \leq K, \\ -\infty & \text{if } k > K. \end{cases}$$

7.3 Regression Strategies

Given class probabilities $\pi_1, \dots, \pi_K$, the regression module maps these to per-class outputs $(\mu_k, \log \sigma_k^2)$ for $k \in [K]$.

7.3.1 Strategy 1: Separate Heads (SEPARATE_HEADS)

Each class k has an *independent* neural network head h_k :

$$h_k : \mathbb{R} \rightarrow \mathbb{R}^2, \quad h_k(\pi_k) = (\mu_k, \log \sigma_k^2).$$

The input to head k is the *scalar* probability $\pi_k(\mathbf{x})$. Heads do not share parameters.

Implementation Note

This is the primary strategy used in benchmarks. The key architectural choice: each head only sees its own class probability, not the full probability vector. The classifier acts as a nonlinear feature extractor projecting onto the probability simplex; each head learns a scalar-to-scalar function.

7.3.2 Strategy 2: Single Head, N Outputs (SINGLE_HEAD_N_OUTPUTS)

One shared neural network takes the full probability vector:

$$h : \mathbb{R}^K \rightarrow \mathbb{R}^{K \times 2}, \quad h(\boldsymbol{\pi}) = \{(\mu_k, \log \sigma_k^2)\}_{k=1}^K.$$

7.3.3 Strategy 3: Single Head, Final Output (SINGLE_HEAD_FINAL_OUTPUT)

One shared network maps probabilities to per-class representations:

$$h : \mathbb{R}^K \rightarrow \mathbb{R}^{K \times 2}, \quad \hat{y} = \sum_{k=1}^K \pi_k \cdot h_k(\boldsymbol{\pi}).$$

Remark 7.1. Strategies 2 and 3 allow parameter sharing across classes but lose the per-class specialization that makes SEPARATE_HEADS effective.

7.4 Law of Total Variance Aggregation

Definition 7.1 (Law of Total Variance Decomposition).

$$\hat{y}(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}] = \sum_{k=1}^K \pi_k(\mathbf{x}) \cdot \mu_k(\mathbf{x}) \quad (11)$$

$$\hat{\sigma}^2(\mathbf{x}) = \text{Var}(Y \mid \mathbf{x}) = \underbrace{\sum_{k=1}^K \pi_k(\mathbf{x}) \cdot \sigma_k^2(\mathbf{x})}_{\text{expected aleatoric variance}} + \underbrace{\sum_{k=1}^K \pi_k(\mathbf{x}) \cdot (\mu_k(\mathbf{x}) - \hat{y}(\mathbf{x}))^2}_{\text{variance of means (epistemic-like)}} \quad (12)$$

Implementation Note

In `pytorch_utils.apply_law_of_total_variance`: input shapes: probabilities (B, K) , per-head outputs $(B, K, 2)$. Output: $(\hat{y}, \log \hat{\sigma}^2)$ with shape $(B, 2)$, where $\log \hat{\sigma}^2 = \log(\max(\hat{\sigma}^2, 10^{-9}))$.

7.5 Gradient Stop Variant

When `optimization_strategy = GRADIENT_STOP`: $\tilde{\pi}_k = \text{stopgrad}(\pi_k)$. The classifier is updated only by cross-entropy; regression heads only by NLL.

7.6 Monotonic Constraints

Separate mean and variance heads with monotonic linear on the mean:

$$\mu_k = g_\mu(\pi_k; \mathbf{w}_\mu) \quad (\text{monotonic}), \quad \log \sigma_k^2 = g_\sigma(\pi_k; \mathbf{w}_\sigma) \quad (\text{unconstrained}). \quad (13)$$

Monotonic linear:

$$f_{\text{mono}}(\mathbf{x}; \mathbf{w}, b) = \begin{cases} \text{softplus}(\mathbf{w})^\top \mathbf{x} + b & \text{positive monotonicity,} \\ -\text{softplus}(\mathbf{w})^\top \mathbf{x} + b & \text{negative monotonicity,} \end{cases}$$

where $\text{softplus}(w) = \log(1 + e^w)$ ensures non-negative weights.

7.7 Boundary Constraints

HARDSIGMOID constrains predictions to $[y_{\min}, y_{\max}]$:

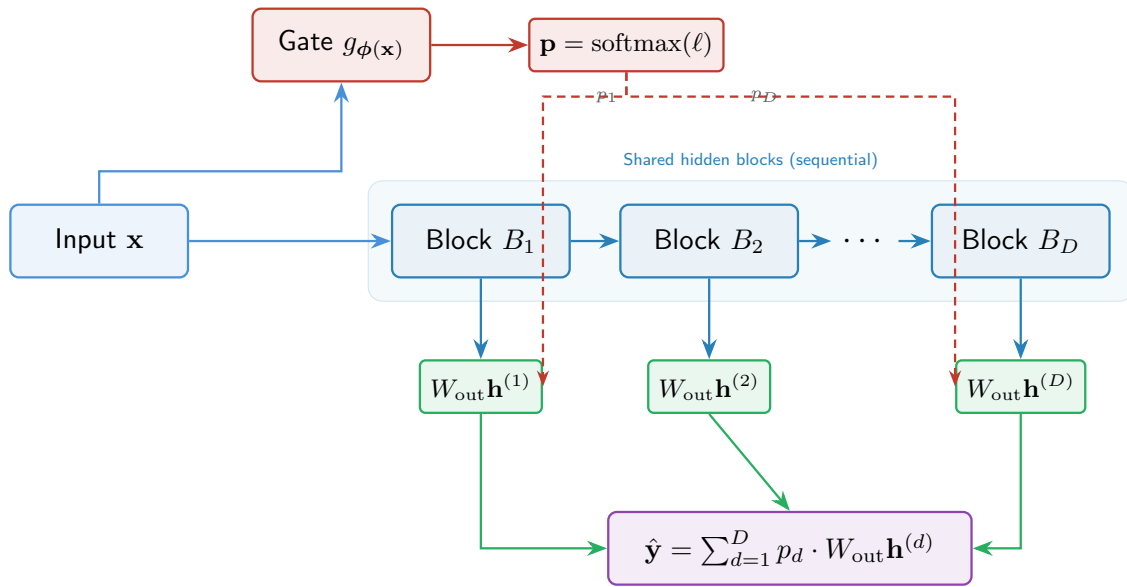
$$\tilde{\mu}_k = y_{\min} + (y_{\max} - y_{\min}) \cdot \text{hardsigmoid}(\mu_k).$$

Uncertainty: Law of total variance (§2.3) — the central novelty of ProbReg.

8 Flexible Neural Network (FlexibleNN)

A neural network with *learnable depth*: a gating subnetwork predicts how many hidden layers each input should use.

8.1 Architecture



Hard inference: $d^* = \arg \max_d p_d$, run only blocks $B_1, \dots, B_{d^*}$

Figure 3: FlexibleNN architecture. Soft forward: all depth paths computed, weighted by p_d . Hard inference: only the argmax depth path is computed per sample.

- **Gate network** $g_{\phi}: \mathbb{R}^d \rightarrow \mathbb{R}^{D_{\max}}$.
- **Hidden blocks** $B_1, \dots, B_{D_{\max}}$: each is Linear $\rightarrow$ BatchNorm $\rightarrow$ Activation $\rightarrow$ Dropout.
- **Output layer** $W_{\text{out}}: \mathbb{R}^h \rightarrow \mathbb{R}^o$, shared across all depths.

8.2 Soft Forward Pass

Depth probabilities: $\mathbf{p} = \text{softmax}(g_{\phi}(\mathbf{x})) \in \Delta^{D_{\max}}$.

For each depth $d \in [D_{\max}]$: $\mathbf{h}^{(d)} = B_d \circ \dots \circ B_1(\mathbf{x})$.

Aggregated output:

$$\hat{\mathbf{y}} = \sum_{d=1}^{D_{\max}} p_d(\mathbf{x}) \cdot W_{\text{out}} \mathbf{h}^{(d)}. \quad (14)$$

Training cost is $O(D_{\max})$ per sample — all depth paths are computed.

8.3 Hard Forward Pass (Inference)

$d^*(\mathbf{x}) = \arg \max_d p_d(\mathbf{x})$. Group samples by depth, run only needed blocks:

Algorithm 1 Hard inference for FlexibleNN

```

1: for  $d = 1$  to  $D_{\max}$  do
2:    $\mathcal{I}_d \leftarrow \{i : d^*(\mathbf{x}_i) = d\}$ 
3:   if  $\mathcal{I}_d \neq \emptyset$  then
4:      $\hat{\mathbf{y}}_{\mathcal{I}_d} \leftarrow W_{\text{out}}(B_d \circ \dots \circ B_1(\mathbf{x}_{\mathcal{I}_d}))$ 
5:   end if
6: end for

```

Compute savings: $(D_{\max} - \bar{d})/D_{\max}$. Accuracy gap: ~ 0.02 MSE.

8.4 Independent-Weights Variant

Each depth option has a *completely independent* network f_d with exactly d hidden layers (no shared blocks):

$$\hat{\mathbf{y}} = \sum_{d=1}^{D_{\max}} p_d(\mathbf{x}) \cdot f_d(\mathbf{x}).$$

This is a **mixture of experts**. More parameters ($O(h^2 \cdot D_{\max}^2/2)$ vs $O(h^2 \cdot D_{\max})$) but full per-depth specialization.

Uncertainty: Same as Plain NN (depends on `uncertainty_method`).

Part III

Dynamic Architecture Selection

The framework below is shared by ProbReg (selecting number of classes K) and FlexibleNN (selecting depth D). We use z as the generic discrete latent variable.

9 Selection Strategies

All strategies compute a probability vector $\mathbf{p} \in \Delta^{|Z|}$ over possible values of z , then aggregate predictions.

9.1 None Strategy (Fixed)

$p_k = \delta_{k, k_{\text{fixed}}}$. No gating network.

9.2 Soft-Gating

Fully differentiable:

$$\mathbf{p} = \text{softmax}(g_{\phi(\mathbf{x})}), \quad \hat{\mathbf{y}} = \sum_k p_k \cdot \hat{\mathbf{y}}^{(k)}.$$

Implementation Note

SoftGating is the best-performing strategy for ProbReg dynamic- K . It achieves MSE matching the best fixed- K baseline with mean $K \approx 2$.

9.3 Gumbel-Softmax

Training: $p_k = \exp((\ell_k + G_k)/\tau) / \sum_j \exp((\ell_j + G_j)/\tau)$, $G_k \sim \text{Gumbel}(0, 1)$.

Evaluation: $p_k = \exp(\ell_k/\tau) / \sum_j \exp(\ell_j/\tau)$ (deterministic).

As $\tau \rightarrow 0$, samples approach one-hot. Aggregation same as Soft-Gating.

Remark 9.1. Gumbel-Softmax for dynamic K has poor empirical performance — the stochastic noise corrupts KL gradients in the ELBO formulation.

9.4 Straight-Through Estimator (STE)

Forward: $\mathbf{p}_{\text{hard}} = \text{one_hot}(\arg \max_k \ell_k)$.

Backward: Gradients flow through the soft distribution: $\mathbf{p}_{\text{STE}} = \mathbf{p}_{\text{hard}} - \text{stopgrad}(\mathbf{p}_{\text{soft}}) + \mathbf{p}_{\text{soft}}$.

Remark 9.2. The weighted-sum form is essential. Overwriting entries of $\mathbf{p}_{\text{soft}}$ with hard one-hot values via indexed assignment severs the gradient path through the gating parameters and makes the strategy equivalent to a frozen argmax.

9.5 REINFORCE

Discrete sampling with policy gradient:

$$k^* \sim \text{Categorical}(\text{softmax}(g_{\phi(\mathbf{x})})), \quad \hat{\mathbf{y}} = \hat{\mathbf{y}}^{(k^*)}.$$

Policy gradient loss:

$$\mathcal{L}_{\text{REINFORCE}} = -\log \pi_{k^*}(\mathbf{x}) \cdot R, \quad R = -\mathcal{L}_{\text{task}}^{\text{detach}}. \quad (15)$$

Remark 9.3. REINFORCE has high variance. Needs baseline subtraction $R \leftarrow R - \bar{R}$ (not yet implemented).

9.6 Strategy Summary

Strategy	Forward Pass	Gradient
None	$p_k = \delta_{k, k_{\text{fixed}}}$	N/A
Soft-Gating	$\mathbf{p} = \text{softmax}(\ell)$	Standard softmax gradient
Gumbel-Softmax	$p_k \propto e^{(\ell_k + G_k)/\tau}$	Reparameterization
STE	Forward: one_hot(arg max); back- ward: soft	Straight-through
REINFORCE	$k^* \sim \text{Cat}(\text{softmax}(\ell))$	$-R \cdot \nabla \log \pi_{k^*}$

10 Regularization for Dynamic Selection

Without regularization, the gating network drifts to $z = z_{\text{max}}$.

10.1 Penalty

Penalize expected complexity:

$$\mathcal{L}_{\text{penalty}} = \lambda \cdot \sum_k k \cdot p_k(\mathbf{x}). \quad (16)$$

Used as K_PENALTY (ProbReg) and DEPTH_PENALTY (FlexibleNN).

10.2 ELBO with KL Divergence

Treat z as a discrete latent variable with prior $p(z)$ favoring simplicity.

Prior:

$$p(z = k) \propto w_k, \quad (w_1, \dots, w_{|\mathcal{Z}|}) = \text{linspace}(3.0, 1.0, |\mathcal{Z}|).$$

The `linspace(3, 1)` parameterization ensures constant steepness regardless of $|\mathcal{Z}|$.

Posterior: $q(z = k | \mathbf{x}) = p_k(\mathbf{x}) = \text{softmax}(g_{\phi(\mathbf{x})})_k$.

ELBO:

$$\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{task}} + \lambda_{\text{KL}} \cdot \text{KL}(q(z | \mathbf{x}) \| p(z)), \quad (17)$$

$$\text{KL}(q \| p) = \sum_k q(z = k | \mathbf{x}) [\log q(z = k | \mathbf{x}) - \log p(z = k)]. \quad (18)$$

10.3 Unified View: ELBO as Occam's Razor

This is the same principle as in variational autoencoders (Kingma & Welling, 2014), but applied to a *discrete architectural choice* rather than a continuous latent code.

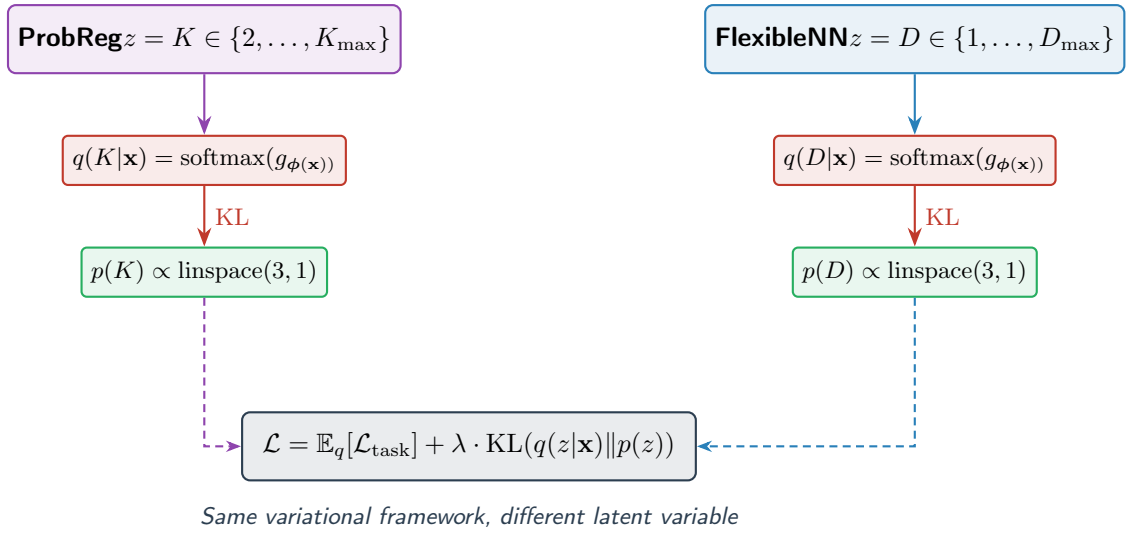


Figure 4: Unified ELBO framework. ProbReg selects K ; FlexibleNN selects D . Both use the same KL penalty against a linspace prior.

10.4 Direct Regression Channel (ProbReg only)

When dynamic K is enabled, index 0 of the gating output corresponds to a *direct regression* bypass: $\hat{\mathbf{y}}^{(0)} = f_{\text{direct}}(\mathbf{x})$, allowing fallback to standard regression when the classification bottleneck adds no value.

Part IV

Loss Functions and Training

11 Loss Functions

11.1 Gaussian Negative Log-Likelihood (NLL)

$$\mathcal{L}_{\text{NLL}}(\hat{y}, \hat{\sigma}^2, y) = \frac{1}{2} \left(\log(2\pi) + \log \hat{\sigma}^2 + \frac{(y - \hat{y})^2}{\hat{\sigma}^2} \right). \quad (19)$$

11.2 β -NLL Loss (Seitzer et al., 2022)

$$\mathcal{L}_{\beta\text{-NLL}} = (\hat{\sigma}_{\text{detach}}^2)^\beta \cdot \frac{1}{2} \left(\log \hat{\sigma}^2 + \frac{(y - \hat{y})^2}{\hat{\sigma}^2} \right). \quad (20)$$

$\beta = 0$: standard NLL. $\beta = 0.5$: recommended default. $\beta = 1$: can over-correct.

Intuition. Large $\hat{\sigma}^2$ amplifies the loss via $(\hat{\sigma}^2)^\beta$, discouraging variance inflation.

11.3 Masked Cross-Entropy Loss

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(\tilde{\ell}_{c^*})}{\sum_j \exp(\tilde{\ell}_j)},$$

where c^* is the true bin index and $\tilde{\ell}$ are masked logits.

11.4 Combined Training Loss (ProbReg)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{regression}} + \lambda_{\text{CE}} \cdot \mathcal{L}_{\text{CE}} + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{regularization}} + \lambda_{\text{boundary}} \cdot \mathcal{L}_{\text{boundary}} + \mathcal{L}_{\text{selection}}. \quad (21)$$

Currently $\lambda_{\text{CE}} = 1$ (implicit, not tuned).

11.5 Boundary Regularization Loss

$$\mathcal{L}_{\text{boundary}} = \frac{1}{N} \sum_{i=1}^N [\max(0, y_{\min} - \hat{y}_i)^2 + \max(0, \hat{y}_i - y_{\max})^2].$$

11.6 Middle-Class Penalty

When K is odd, penalize the middle class $k_{\text{mid}} = \lceil K/2 \rceil$:

$$\mathcal{L}_{\text{mid}} = -\log \mathcal{N}(\mu_{k_{\text{mid}}}; \bar{y}_{\text{mid}}, \sigma_{k_{\text{mid}}}^2).$$

11.7 Tree-Model NLL Objective

See §4.2 (in the GBT section) for the custom gradients and Hessians used by XGBoost, LightGBM, and CatBoost.

12 Learned Regularization

Instead of fixing λ_1, λ_2 , they are learned via separate optimizer steps. Let d be the number of model parameters.

12.1 L1 Only (Laplace Prior)

$$\mathcal{L}_{\text{reg}} = -d \cdot \log \frac{\lambda_1}{2} + \lambda_1 \|\mathbf{w}\|_1, \quad \lambda_1 = e^{\ell_1}.$$

12.2 L2 Only (Gaussian Prior)

$$\mathcal{L}_{\text{reg}} = -\frac{d}{2} \log \frac{\lambda_2}{\pi} + \lambda_2 \|\mathbf{w}\|_2^2.$$

12.3 L1 + L2 (Horseshoe-like Prior)

$$\mathcal{L}_{\text{reg}} = d \cdot \log Z + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2, \quad \log Z = \frac{1}{2} \log \frac{\pi}{\lambda_2} + \frac{\lambda_1^2}{4\lambda_2} + \log \text{erfc}\left(\frac{\lambda_1}{2\sqrt{\lambda_2}}\right).$$

13 Target Transforms

13.1 Symmetric Log Transform (Symlog)

For targets spanning multiple orders of magnitude:

$$\text{symlog}(x) = \text{sign}(x) \cdot \log(1 + |x|), \quad (22)$$

$$\text{symexp}(y) = \text{sign}(y) \cdot (\exp(|y|) - 1). \quad (23)$$

symexp is the exact inverse. Compresses large values ($\text{symlog}(1000) \approx 6.9$) while preserving sign.

13.2 Uncertainty Conversion: Monte Carlo

If the model predicts (μ_s, σ_s^2) in symlog space:

1. Sample $\epsilon_1, \dots, \epsilon_M \sim \mathcal{N}(\mu_s, \sigma_s^2)$, $M = 200$.
2. Apply: $z_m = \text{symexp}(\epsilon_m)$.
3. Return: $\hat{\mu} = \frac{1}{M} \sum z_m$, $\hat{\sigma} = \sqrt{\frac{1}{M} \sum (z_m - \hat{\mu})^2}$.
Exact up to MC variance; handles nonlinearity of symexp near zero.

14 Training Loop

All neural models follow this loop (`base_pytorch.py`):

Algorithm 2 Training loop for PyTorch models

Require: Model f_{θ} , data $(\mathbf{X}, \mathbf{y})$, learning rate η , patience P

```

1: Split into train/validation
2: Initialize Adam optimizer
3: best_loss  $\leftarrow \infty$ , wait  $\leftarrow 0$ 
4: for epoch = 1 to max_epochs do
5:   for each mini-batch  $(\mathbf{X}_b, \mathbf{y}_b)$  do
6:      $\hat{\mathbf{y}}_b \leftarrow f_{\theta}(\mathbf{x}_b)$ 
7:      $\mathcal{L} \leftarrow \text{criterion}(\hat{\mathbf{y}}_b, \mathbf{y}_b) + \mathcal{L}_{\text{reg}}(\theta)$ 
8:     Backprop and update  $\theta$  via Adam
9:     if learned regularization then update  $\lambda$  params separately
10:    end if
11:  end for
12:   $\mathcal{L}_{\text{val}} \leftarrow$  evaluate on validation set
13:  if  $\mathcal{L}_{\text{val}} < \text{best\_loss}$  then
14:    Save checkpoint, wait  $\leftarrow 0$ 
15:  else
16:    wait  $\leftarrow$  wait + 1
17:    if wait  $\geq P$  then break
18:    end if
19:  end if
20: end for
21: Load best checkpoint

```

Part V

Baseline Models

15 NGBoost (Duan et al., 2020)

Natural gradient boosting. For $\mathcal{N}(\mu, \sigma^2)$, update at each step uses the *natural gradient*:

$$\tilde{\nabla}_{\boldsymbol{\theta} \mathcal{L} = \mathcal{I}(\boldsymbol{\theta})^{-1} \nabla_{\boldsymbol{\theta} \mathcal{L}}, \quad \mathcal{I} = \begin{pmatrix} 1/\sigma^2 & 0 \\ 0 & 1/(2\sigma^4) \end{pmatrix}.$$

This accounts for the geometry of the parameter space, preventing over-aggressive σ updates.

Uncertainty: Learned $\hat{\sigma}^2(\mathbf{x})$, input-dependent via tree structure. Primary tree-based competitor to ProbReg.

16 Mixture Density Network — MDN (Bishop, 1994)

A neural network outputting M -component Gaussian mixture parameters:

$$p(y | \mathbf{x}) = \sum_{m=1}^M w_m(\mathbf{x}) \cdot \mathcal{N}(y; \mu_m(\mathbf{x}), \sigma_m^2(\mathbf{x})), \quad (24)$$

with $w_m = \text{softmax}(\ell^w)_m$, $\sigma_m = \exp(\ell^\sigma_m/2)$.

Loss: $\mathcal{L}_{\text{MDN}} = -\frac{1}{N} \sum_i \log(\sum_m w_m(\mathbf{x}_i) \cdot \phi(y_i; \mu_m, \sigma_m^2))$.

Uncertainty: Full mixture variance via law of total variance: $\hat{\sigma}^2 = \sum_m w_m \sigma_m^2 + \sum_m w_m (\mu_m - \hat{\mu})^2$.

vs ProbReg: MDN has no structural constraint on components; ProbReg imposes a classification bottleneck that regularizes the mixture.

17 Deep Ensembles (Lakshminarayanan et al., 2017)

Train M independently initialized networks, each outputting (μ_m, σ_m^2) :

$$\hat{\mu} = \frac{1}{M} \sum_{m=1}^M \mu_m, \quad \hat{\sigma}^2 = \frac{1}{M} \sum_{m=1}^M \sigma_m^2 + \frac{1}{M} \sum_{m=1}^M (\mu_m - \hat{\mu})^2. \quad (25)$$

Same law-of-total-variance decomposition as ProbReg with $\pi_m = 1/M$.

vs ProbReg: M full-size networks ($O(M)$ forward passes) vs lightweight per-class heads behind a shared classifier ($O(1)$ forward passes).

18 Quantile Regression NN (Koenker, 2005)

Trained with pinball loss at multiple quantiles $\tau_1 < \dots < \tau_Q$:

$$\mathcal{L}_{\text{QR}} = \frac{1}{NQ} \sum_{i,j} L_{\tau_j}(y_i, \hat{q}_{\tau_j}(\mathbf{x}_i)), \quad L_{\tau}(y, \hat{q}) = \begin{cases} \tau(y - \hat{q}) & y \geq \hat{q}, \\ (1 - \tau)(\hat{q} - y) & y < \hat{q}. \end{cases}$$

Distribution-free. Intervals: $[\hat{q}_{\alpha/2}, \hat{q}_{1-\alpha/2}]$.

19 Conformalized Quantile Regression (Romano et al., 2019)

CQR calibrates the intervals produced by a quantile regressor (§18) via split conformal prediction (§25) so that coverage is distribution-free and finite-sample-valid.

Fit $(\hat{q}_{\alpha/2}, \hat{q}_{1-\alpha/2})$ on the training fold. On a held-out calibration set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ compute the conformity scores

$$R_i = \max(\hat{q}_{\alpha/2}(\mathbf{x}_i) - y_i, y_i - \hat{q}_{1-\alpha/2}(\mathbf{x}_i)), \quad (26)$$

which measure how far each target falls outside the predicted interval (negative when inside). Take the finite-sample-corrected quantile

$$\hat{Q}_\alpha = \text{Quantile}\left(\{R_i\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right).$$

Calibrated interval:

$$C(\mathbf{x}) = [\hat{q}_{\alpha/2}(\mathbf{x}) - \hat{Q}_\alpha, \hat{q}_{1-\alpha/2}(\mathbf{x}) + \hat{Q}_\alpha].$$

Coverage $\Pr(y \in C(\mathbf{x})) \geq 1 - \alpha$ holds under exchangeability. Unlike marginal split conformal (§25), the CQR interval width varies with $\mathbf{x}$ because the base quantile regressor is heteroscedastic.

20 Gaussian Process

$$\hat{\mu}(\mathbf{x}^*) = \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \quad (27)$$

$$\hat{\sigma}^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*. \quad (28)$$

Training $O(N^3)$; limited to $N \leq 5000$.

21 Constant-Variance NN

$f_{\theta(\mathbf{x})=(\mu(\mathbf{x}), \log \sigma^2)}$ where $\log \sigma^2$ is a single global parameter.

Implementation Note

Implemented as ProbReg with $K = 1$, `uncertainty_method = PROBABILISTIC`.

22 FT-Transformer (Gorishniy et al., 2021)

Tabular transformer: tokenizes each feature into an embedding and applies multi-head self-attention.

Feature tokenization: For feature j : $\mathbf{e}_j = \mathbf{w}_j \cdot x_j + \mathbf{b}_j \in \mathbb{R}^{d_{\text{emb}}}$. Prepend [CLS] token $\mathbf{e}_0$.

Transformer blocks (L layers):

$$\tilde{\mathbf{E}}^{(l)} = \mathbf{E}^{(l-1)} + \text{MultiHeadAttn}(\text{LN}(\mathbf{E}^{(l-1)})), \quad (29)$$

$$\mathbf{E}^{(l)} = \tilde{\mathbf{E}}^{(l)} + \text{FFN}(\text{LN}(\tilde{\mathbf{E}}^{(l)})). \quad (30)$$

Prediction: $\hat{y} = \mathbf{w}_{\text{out}}^\top \mathbf{e}_0^{(L)} + b_{\text{out}}$.

Uncertainty: No native UQ. Can combine with Deep Ensembles, MC Dropout, or conformal prediction.

Proposed extension (Paper C): Apply FlexibleNN depth gating to FT-Transformer blocks — variable self-attention depth per input with ELBO regularization.

23 TabPFN (Hollmann et al., 2024)

Pre-trained transformer performing in-context learning:

$$\hat{y}(\mathbf{x}^*) = \text{TabPFN}(\mathbf{x}^*, \mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}).$$

No task-specific training. Pre-trained on millions of synthetic datasets.

Uncertainty: Outputs a predictive distribution (quantile predictions for intervals).

Limits: $N \leq 10,000$, $d \leq 500$.

24 Laplace-Approximation BNN (Daxberger et al., 2021)

Not yet implemented — *included for comparison completeness*. Post-hoc Bayesian: train MAP estimate $\boldsymbol{\theta}_{\text{MAP}}$, approximate posterior as Gaussian: $p(\boldsymbol{\theta} \mid \mathcal{D}) \approx \mathcal{N}(\boldsymbol{\theta}_{\text{MAP}}, (\nabla^2 \mathcal{L})^{-1})$.

Predictive variance via linearization:

$$\hat{\sigma}^2(\mathbf{x}^*) = \sigma_n^2 + \mathbf{J}(\mathbf{x}^*)^\top (\nabla^2 \mathcal{L})^{-1} \mathbf{J}(\mathbf{x}^*), \quad (31)$$

where $\mathbf{J}(\mathbf{x}^*) = \nabla_{\boldsymbol{\theta}} f(\mathbf{x}^*; \boldsymbol{\theta}_{\text{MAP}})$.

Available via `laplace-torch` (~50 lines wrapper).

25 Conformal Prediction

Distribution-free intervals that wrap *any* fitted model. Given a calibration set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and miscoverage α .

25.1 Split Conformal (Marginal)

1. Nonconformity scores: $R_i = |y_i - \hat{f}(\mathbf{x}_i)|$.
2. Finite-sample-corrected quantile:

$$\hat{q}_\alpha = \text{Quantile} \left(\{R_i\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right). \quad (32)$$

3. Prediction interval: $C(\mathbf{x}) = [\hat{f}(\mathbf{x}) - \hat{q}_\alpha, \hat{f}(\mathbf{x}) + \hat{q}_\alpha]$.

Definition 25.1 (Coverage Guarantee). Under exchangeability: $\Pr(y \in C(\mathbf{x})) \geq 1 - \alpha$.

Interval width is a constant $2\hat{q}_\alpha$ — over-covers in low-noise regions and under-covers in high-noise regions on a per-point basis.

25.2 Locally-Adaptive Conformal (Lei & Wasserman, 2014)

Given a base model that supplies both a point prediction $\hat{f}(\mathbf{x})$ and an uncertainty estimate $\hat{\sigma}(\mathbf{x})$, normalize the nonconformity scores by the predicted standard deviation:

$$\tilde{R}_i = \frac{|y_i - \hat{f}(\mathbf{x}_i)|}{\hat{\sigma}(\mathbf{x}_i)}, \quad \hat{Q}_\alpha = \text{Quantile} \left(\{\tilde{R}_i\}, \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right). \quad (33)$$

Adaptive interval:

$$C(\mathbf{x}) = [\hat{f}(\mathbf{x}) - \hat{Q}_\alpha \cdot \hat{\sigma}(\mathbf{x}), \hat{f}(\mathbf{x}) + \hat{Q}_\alpha \cdot \hat{\sigma}(\mathbf{x})].$$

The same coverage guarantee $\Pr(y \in C(\mathbf{x})) \geq 1 - \alpha$ holds, but intervals now contract where the base model is confident and widen where it is uncertain. This gives *conditional* coverage if $\hat{\sigma}(\mathbf{x})$ tracks the true residual scale.

Part VI

Evaluation

26 Evaluation Metrics

26.1 Point-Prediction Metrics

$$\text{MAE} = \frac{1}{N} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_i (y_i - \hat{y}_i)^2}, \quad (34)$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad \text{MAPE} = \frac{100}{N} \sum_i \frac{|y_i - \hat{y}_i|}{|y_i|}. \quad (35)$$

26.2 Proper Scoring Rules

Negative Log-Likelihood:

$$\text{NLL} = \frac{1}{N} \sum_i \frac{1}{2} \left(\log(2\pi) + \log \hat{\sigma}_i^2 + \frac{(y_i - \hat{y}_i)^2}{\hat{\sigma}_i^2} \right).$$

CRPS. For $\mathcal{N}(\mu, \sigma^2)$:

$$\text{CRPS} = \sigma \left[\frac{y - \mu}{\sigma} \left(2\Phi\left(\frac{y - \mu}{\sigma}\right) - 1 \right) + 2\phi\left(\frac{y - \mu}{\sigma}\right) - \frac{1}{\sqrt{\pi}} \right]. \quad (36)$$

Strictly proper. Generalizes to non-Gaussian predictive distributions via numerical integration.

Pinball (quantile) loss. $L_\tau(y, \hat{q}_\tau) = \max(\tau(y - \hat{q}_\tau), (1 - \tau)(\hat{q}_\tau - y))$; at $\tau = 0.5$ equals MAE.

26.3 Interval Metrics

PICP. $\text{PICP}_\alpha = \frac{1}{N} \sum_i \mathbf{1}[y_i \in [\hat{y}_i - z_{\alpha/2} \hat{\sigma}_i, \hat{y}_i + z_{\alpha/2} \hat{\sigma}_i]]$.

MPIW. $\text{MPIW}_\alpha = \frac{1}{N} \sum_i (U_i - L_i)$.

Winkler score. For interval $[L, U]$ at level $(1 - \alpha)$:

$$W_\alpha = (U - L) + \frac{2}{\alpha} (L - y) \mathbf{1}[y < L] + \frac{2}{\alpha} (y - U) \mathbf{1}[y > U]. \quad (37)$$

Strictly proper scoring rule for intervals.

26.4 Calibration via PIT (Kuleshov et al., 2018)

Probability Integral Transform: $U_i = F(y_i \mid \mathbf{x}_i)$. If the predictive distribution is calibrated, $U \sim \text{Uniform}(0, 1)$.

Calibration curve: for target quantiles $p_0 < \dots < p_B \in [0, 1]$, $\hat{p}_j = \frac{1}{N} \sum_i \mathbf{1}[U_i \leq p_j]$.

Regression ECE: $\text{ECE} = \frac{1}{B+1} \sum_j |p_j - \hat{p}_j|$.

Miscalibration area: $\int_0^1 |\hat{p}(t) - t| dt$ (trapezoidal). More robust than bin-averaged ECE.

Sharpness: $\frac{1}{N} \sum_i \hat{\sigma}_i$. Always reported jointly with calibration — a calibrated model with wide intervals is useless.

26.5 Post-hoc Recalibration

Isotonic regression from predicted CDF values to empirical CDF on a held-out set (Kuleshov et al., 2018). Let $\{U_i^{\text{cal}}\}$ be PIT values on the calibration set. Fit a monotone $\hat{R} : [0, 1] \rightarrow [0, 1]$ with

$$\hat{R}(U_{(i)}^{\text{cal}}) \approx i/n_{\text{cal}}.$$

At test time apply $U_i^{\text{recal}} = \hat{R}(U_i)$. Report metrics as “+recal” variants to separate raw calibration quality from post-hoc-fixable calibration quality.

26.6 PIT Uniformity Tests

Sharper than visual inspection of histograms. Let $\{U_i\}$ be the PIT values.

Kolmogorov-Smirnov: $\text{KS} = \sup_t |\hat{F}_N(t) - t|$ where $\hat{F}_N$ is the empirical CDF of $\{U_i\}$.

Cramér-von Mises: $\text{CvM} = \frac{1}{12N} + \sum_{i=1}^N \left(\frac{2i-1}{2N} - U_{(i)} \right)^2$. More sensitive to global deviations.

Anderson-Darling: weights deviations by $1/[t(1-t)]$, emphasizing the tails of the distribution. Implemented by transforming $Z_i = \Phi^{-1}(U_i)$ and applying the standard normal AD test, since SciPy does not expose AD directly for the uniform distribution.

27 Photometric Redshift Metrics

Normalized residual: $\Delta z_i = (z_{\text{phot},i} - z_{\text{spec},i}) / (1 + z_{\text{spec},i})$.

Bias: $\langle \Delta z \rangle$. **Scatter:** $\sigma_{\text{MAD}} = 1.4826 \cdot \text{median}(|\Delta z_i - \text{median}(\Delta z)|)$. **Outlier fraction:** $\eta = \frac{1}{N} \sum_i \mathbf{1}[|\Delta z_i| > 0.15]$.

PIT uniformity: KS, CvM, and AD tests on $\{F(z_{\text{spec},i} | \mathbf{x}_i)\}$ vs Uniform(0, 1) (§26.4). CvM and AD are more sensitive to tail deviations, which drive outlier rates.

CDE loss (Izbicki & Lee, 2017):

$$\text{CDE_loss} = -\frac{1}{N} \sum_i \log \hat{p}(z_{\text{spec},i} | \mathbf{x}_i), \quad (38)$$

the negative log-likelihood of the true redshift under the model’s predicted conditional density. Standard metric for full-PDF quality beyond point estimates.